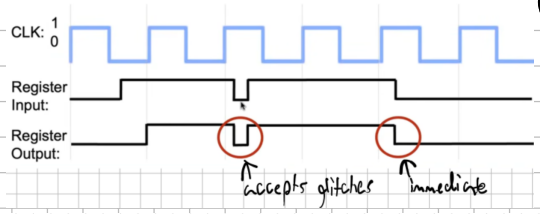
→ we use the clock to trigger transitions across many elements in sync. circuit

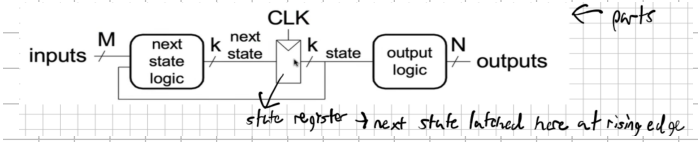
→ during a clock cycle, combinational logic blocks compute the next state (also known as next-state logic)

→ in the timing chapter, you will see how important it is for a clock cycle to be longer than any combinational (computing) delay

Reason why we use flip-flops and not latches in sequential circuits:



(This is for a D latch)



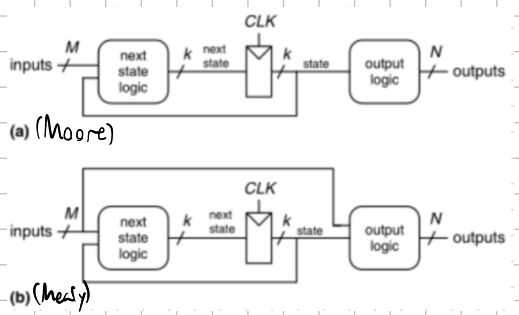
← parts

FSM: discrete time-model of a stateful system

it contains:
- a finite number of states
- a finite number of external inputs
- a finite number of external outputs
- a specification of all state transitions
- a specification of how external outputs are determined

→ The state register(s) store the current state and output the next state at the rising edge (only sequential circuit)

→ Combinational circuits: Next state logic and output logic



Moore machines: outputs depend on current state only

Mealy machines: outputs also depend on current inputs

FSM design (crucial for exam!):

1. Identify your inputs/outputs
2. Draw your state transition diagram
   - ↳ start with the reset state and what is caused by it
   - → add states and transitions
   - → this is similar to programming
     - → FSMs have a sequential control flow (conditionals, jumps)
     - → if/else construct controlled by inputs → outputs controlled by state or inputs
3. Moore machine: write state transition table, write output table

   Mealy machine: write state transition and output table (combined)
4. Select state encodings:

   Binary (full) encoding: Use minimum possible number of bits → $\log_2$(#states) bits
   → Example: 00, 01, 10, 11
   → minimizes # flip-flops, not necessarily output logic

   One-hot encoding: one bit for every state → #states bits needed → example: 0001, 0010, 0100, 1000
   → simple, automatable, doesn't need much next state logic, but inefficient for flip-flops

   Output encoding: directly encode outputs in state

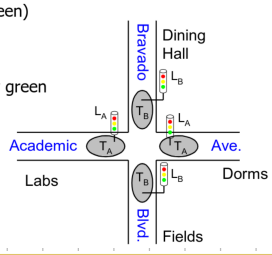
→ example: 001 (green), 010 (yellow), 100 (red), 110 (red/yellow) for a traffic light (only works for Moore type)

5. Write Boolean equations for next state and output logic

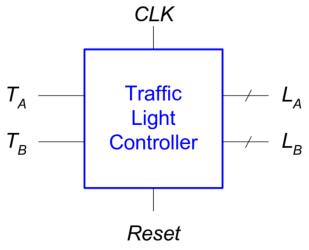
6. Sketch the circuit (this is formal, you won't have to do this)

Simple example:

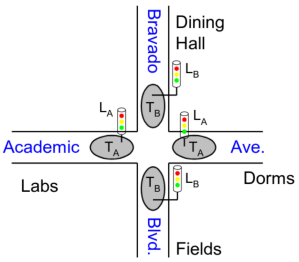
- "Smart" traffic light controller
  - **2 inputs:**
    - Traffic sensors: T_A, T_B (TRUE when there's traffic)
  - **2 outputs:**
    - Lights: L_A, L_B (Red, Yellow, Green)
  - State changes every 5 seconds
    - Except if green and traffic, stay green

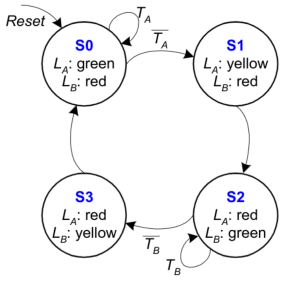


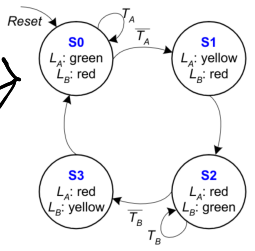
- **Inputs:** CLK, Reset, T_A, T_B
- **Outputs:** L_A, L_B



- **Moore FSM:** outputs labeled in each state
  - **States:** Circles
  - **Transitions:** Arcs

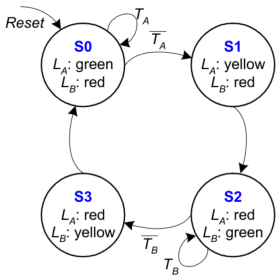






Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

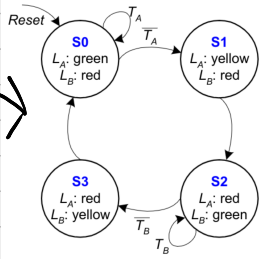
State	Encoding
S0	00
S1	01
S2	10
S3	11



Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

State	Encoding
S0	00
S1	01
S2	10
S3	11

$$S'_1 = (\overline{S_1} \cdot S_0) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B}) + (S_1 \cdot \overline{S_0} \cdot T_B)$$

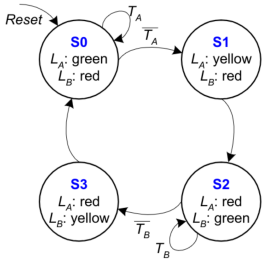


Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

State	Encoding
S0	00
S1	01
S2	10
S3	11

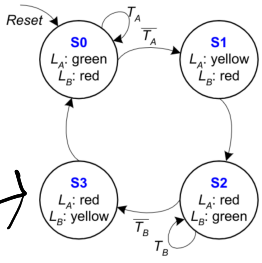
$S'_1 = S_1 \text{ xor } S_0$ **(Simplified)**

$$S'_0 = (\overline{S_1} \cdot \overline{S_0} \cdot \overline{T_A}) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B})$$



Current State		Outputs	
S_1	S_0	L_A	L_B
0	0	green	red
0	1	yellow	red
1	0	red	green
1	1	red	yellow

Output	Encoding
green	00
yellow	01
red	10

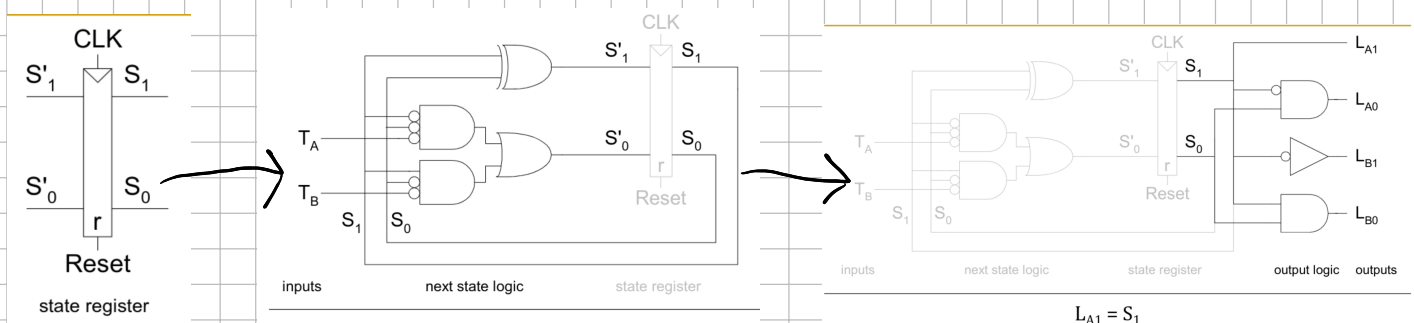


Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

$$L_{A1} = S_1$$
$$L_{A0} = \overline{S_1} \cdot S_0$$
$$L_{B1} = \overline{S_1}$$
$$L_{B0} = S_1 \cdot S_0$$

Output	Encoding
green	00
yellow	01
red	10

Schematic:



$S'_1 = S_1 \text{ xor } S_0$

$S'_0 = (\overline{S_1} \cdot \overline{S_0} \cdot \overline{T_A}) + (S_1 \cdot \overline{S_0} \cdot \overline{T_B})$

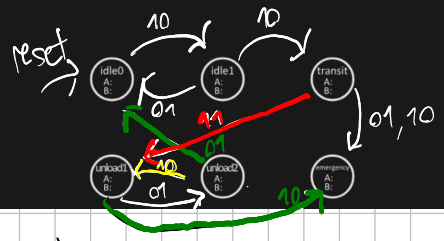
$L_{A1} = S_1$
$L_{A0} = \overline{S_1} \cdot S_0$
$L_{B1} = \overline{S_1}$
$L_{B0} = S_1 \cdot S_0$

Practical Exam Example:

Note: I see it as very unlikely that you will have to draw up a schematic, everything else is important!

from F522:

(a) [25 points] Complete the Moore-type FSM below by (1) drawing the transition edges between the states (including reset), (2) specifying the edges' respective input bits, and (3) specifying the output bits of each state. Any input for which no outgoing edge is specified is assumed as a self loop. The 6 given states are sufficient, do not draw additional states.
Note: Passengers sometimes slip in or out through incorrect doors, or even through closed doors. Your FSM must correctly handle such cases.



outputs: idle 0 → 10, idle 1 → 10, transit → 00, unload1 → 01, unload2 → 01, emergency → 11

2 Finite State Machines [50 points]

The Polybahn from Central to Polyterasse has broken down! To fix it you need to design a finite state machine that controls the Polybahn's two doors A and B.

The Polybahn should operate as follows:

- Initially the Polybahn is empty and **idle**, and passengers can enter through door A.
- The Polybahn is full when it carries 2 passengers.
- When it is full and idle, the Polybahn goes into **transit** to the other station with both doors closed.
- After reaching the station, the Polybahn will **unload** all passengers through door B, while door A is still closed.
- After the last passenger has exited the Polybahn, door B closes and the Polybahn becomes **idle**.
- Should (1) a passenger fall out of the Polybahn during **transit**, or (2) the Polybahn become overfull (≥ 3 passengers) at any point, it stops in **emergency** mode, where it opens all doors and remains (unless reset to the initial **idle** state).

The FSM receives two input bits, with the following meaning:

Input	Meaning
00	no change
01	exactly one passenger left
10	exactly one passenger entered
11	the Polybahn arrived in a station

The FSM produces two output bits: The first bit, A, holds door A open when it is 1. The second bit, B, holds door B open when it is 1.

(a) [25 points] Complete the Moore-type FSM below by (1) drawing the transition edges between the states (including reset), (2) specifying the edges' respective input bits, and (3) specifying the output bits of each state. Any input for which no outgoing edge is specified is assumed as a self loop. The 6 given states are sufficient, do not draw additional states.
Note: Passengers sometimes slip in or out through incorrect doors, or even through closed doors. Your FSM must correctly handle such cases.

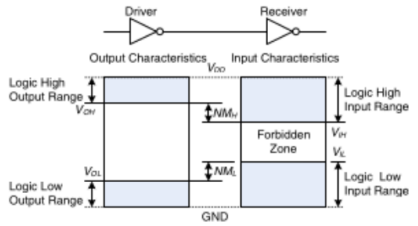
Timing:

Electrical Basics:

lowest voltage in a system → ground/GND (usually 0V)

highest voltage in a system → V_{DD}, comes from PSU

Logic Levels:



→ first gate: driver, second gate: receiver

→ driver produces logical LOW (0) → output between 0 and V_{OL} (output low)

HIGH (0) → output between V_{OH} (output high) and V_{DD}

→ receiver gets input between 0 and V_{IL} (input low) → reads logical LOW

V_{IH} (input high) and V_{DD} → reads logical HIGH

→ forbidden zone: between V_{IL} and V_{IH} (caused by defects or noise)

→ Noise Margins: NM_L (low margin) $= V_{IL} - V_{OL}$ → reasoning: we need output high > input high, output low < input low to correctly interpret outputs (with noise)

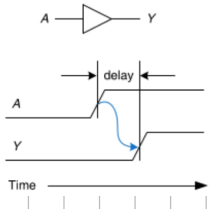
NM_H (high margin) $= V_{OH} - V_{IH}$

→ Static Discipline: to avoid forbidden zone → given a log. valid input, any circuit element produces log. valid outputs

→ we can arbitrarily choose V_{DD}, but communicating gates must have compatible logic levels

→ we group gates into logic families like CMOS → all gates in CMOS adhere to static discipline with each other

Combinational Timing: we want to optimize the speed of a circuit



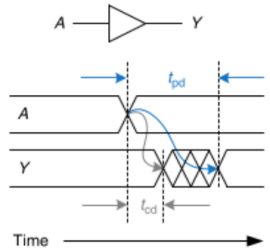
→ input change and output change have a delay for any circuit element

→ timing diagrams like this show the transient response of circuits

→ rising edge → logic LOW → HIGH, falling edge → logic HIGH → LOW

→ blue arrow indicates that rising edge of A causes rising edge of Y

→ delay is measured from 50% point (when signal is 50% between LOW/HIGH) of input to 50% point of output



t_{pd} → propagation delay → max. time from the input change to the output's final value

t_{cd} → contamination delay → max. time from the input change to the output value change

Critical path: slowest path with the most changes

Short path: fastest path with the least changes

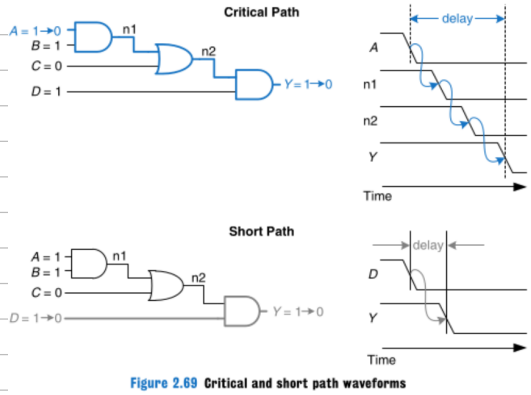


Figure 2.69 Critical and short path waveforms

→ propagation delay of combinational circuit: sum of t_{pd} on critical path

→ contamination delay of comb. circuit: sum of t_{cd} on short path

Glitches: single input transition causes multiple output transitions

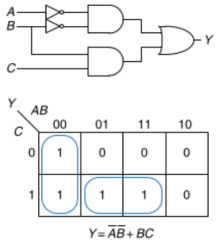
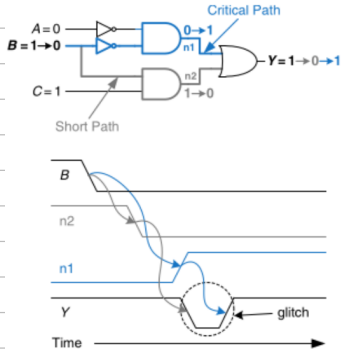


Figure 2.75 Circuit with a glitch



→ short path goes through AND and OR

→ critical goes through AND, OR, NOT

→ falling edge on B → n2 falls on short path before n1 rises on crit.

→ until n1 rises, two inputs to OR gate are 0 → Y drops to 0

→ when n1 does rise → Y comes back to 1 → that was our glitch

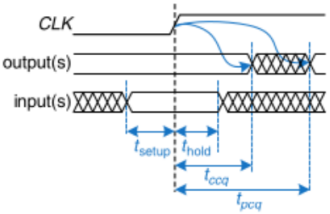
→ glitches are fine when we wait for t_{pd} before using the output, but here, we could prevent it by adding a redundant gate

Sequential Timing: remember how in a flip-flop, D is sampled on the clock edge?

→ an apt comparison to what happens in circuits is how you have to hold a camera still for a while before a shot and after (aperture time) → our aperture time consists of the setup (t_{setup}) and hold (t_{hold}) time

→ sequential circuits have a clock-to-Q contamination (t_{ccq}) and propagation (t_{pcq}) delay → similar to combinational circuits, they are the fastest and slowest delays

→ for the circuit to sample its input correctly, the input must have been stable for t_{setup}

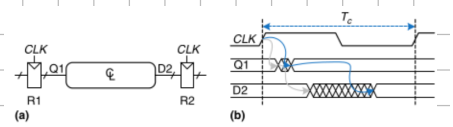


before the clock edge and t_{hold} after $\Rightarrow$ aperture time = $t_{setup} + t_{hold}$

Dynamic Discipline: the inputs of a sync. circuit must be stable during the aperture time

→ sample inputs that are not changing

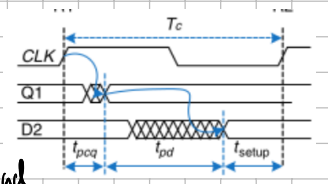
System Timing: clock period T_c = time between rising edges of a clock, $1/T_c = f_c$ → clock frequency, which is measured in Hz (cycles per second)



→ in this image, we try to calculate the clock period

→ R_1 produces Q_1 on the rising edge, they go through a block of comb. logic that produces D_2, which goes to R_2

→ gray arrows → contamination delay, blue arrows → propagation delay



Setup Time Constraint:

→ D_2 must be settled no later than before t_{setup}

→ we know that $T_c \geq t_{pcq} + t_{pd} + t_{setup}$ → can be rearranged to $t_{pd} \leq T_c - (t_{pcq} + t_{setup})$ (sequencing overhead)

this equation is called setup time constraint,

because the comb. logic block cannot meaningfully compute D_2 during t_{pcq} and t_{setup}

→ if t_{pd} is too long, D_2 may not be at its final value when needed → malfunction → solution: increase T_c or redesign CL

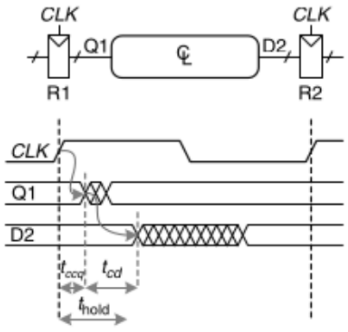
Hold Time Constraint:

→ the designer usually doesn't have control over t_{ccq}, t_{pcq}, t_{hold}, t_{setup}, or T_c (depends on manufacturers; dictated internally)

→ hold time constraint: D_2 must not change for t_{hold} after rising edge, but it can change $t_{ccq} + t_{cd}$ after rising edge $\Rightarrow t_{ccq} + t_{cd} \geq t_{hold}$

$\Rightarrow t_{cd} \geq t_{hold} - t_{ccq}$ (min-delay constraint)

↑ t_{ccq} and t_{hold} outside our control



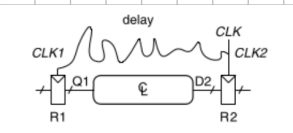
Example: 2 flip-flops: no comb. logic between → $t_{hold} \leq t_{ccq}$, in practice, $t_{hold} = 0$ (or any chain of seq. logic elements)

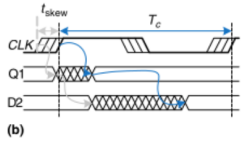
→ hold time violations cannot be fixed without redesigning the circuit (by increasing contamination delay) → cannot be fixed by adjusting clock period → very serious for real circuits

Clock skew (this came up in the lecture, so we must examine it)

→ clock edges may vary (clock skew), caused by differing clock-register wire length, noise, and using both gated and ungated clocks

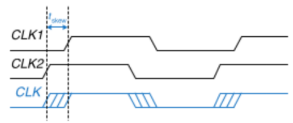
→ here, CLK_2 is early because of the routing of the clock wire between R_1 and R_2



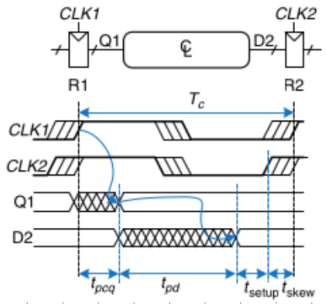


→ clock may arrive up to t_{skew} earlier

→ effect on setup time constraint:



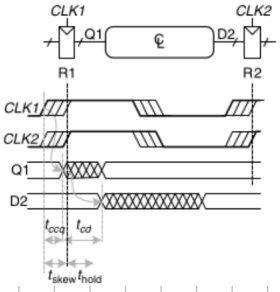
→ worst case: R_1 receives latest skewed clock, R_2 receives earliest skewed clock because comb. logic propagation has least time



→ data still has to setup and propagate before sampling $\Rightarrow T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew}$

$\Rightarrow t_{pd} \leq T_c - (t_{pcq} + t_{setup} + t_{skew})$ → skew added to constraint

Effect on hold time constraint:



→ worst case: R_1 gets earliest skewed clock, R_2 receives latest skewed clock

$\Rightarrow$ data must not arrive until hold time after late clock

$\Rightarrow t_{ccq} + t_{cd} \geq t_{hold} + t_{skew} \Rightarrow t_{cd} \geq t_{hold} + t_{skew} - t_{ccq}$

→ overall, clock skew increases both setup and hold time

Metastability: result of violating dyn. discipline (in flip-flops) $\Rightarrow$ output can take on voltage between 0 and V_{DD} in forbidden zone → metastable state

→ flip-flop will resolve output to 1 or 0, but resolution time is unbounded

Resolution time: when a flip-flop input changes at random time during clock cycle, t_{res} (resolution time is a random variable) (AHW flashbacks! )

→ input changes outside aperture → $t_{res} = t_{pcq}$, input changes within aperture → t_{res} can be longer)

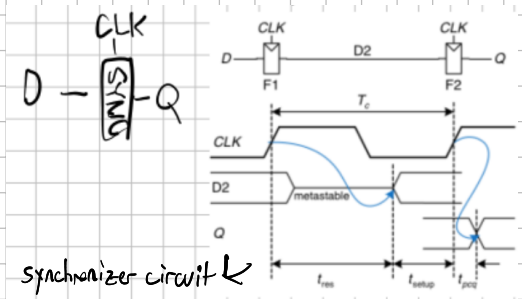
Equation for t substantially longer than t_{pcq} → $P(t_{res} > t) = \frac{T_o}{T_c} e^{-\frac{t}{\tau}}$ → T_o and τ are characteristic to flip-flop

→ $\frac{T_o}{T_c}$ → prob. that input changes at a bad time

Synchronizers:

→ asynchronous inputs like human inputs are inevitable in larger systems → needs handling to prevent metastability

→ a synchronizer receives async. input D and CLK, produces Q in bounded time, the output has a valid logic level with near certainty (if D stable, Q = D, else choose HIGH or LOW)



synchronizer circuit ↑

→ in the image, F_1 samples D on the rising edge of CLK (during aperture, momentarily metastable)

→ Over the clock period, D2 resolves, F_2 samples D2, we get a good output Q

→ synchronizer fails when Q becomes metastable → $t_{res} > T_c - t_{setup}$

→ $P(\text{failure}) = \frac{T_o}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}}$, $P(\text{failure})/\text{sec} = N \frac{T_o}{T_c} e^{-\frac{T_c - t_{setup}}{\tau}}$ (N: times D changes per second)

(MTBF)

→ mean time between failures usually measures system reliability $\Rightarrow MTBF = \frac{1}{P(\text{failure/sec})} = \frac{T_c e^{\frac{T_c - t_{setup}}{\tau}}}{N T_o}$

Some notes on parallelism (pipelining will be introduced)

Token → group of inputs processed to produce group of outputs

Latency → time for 1 token to pass from start to end

Throughput → number of tokens per unit time

Spatial parallelism: multiple copies of hardware, Temporal parallelism (pipelining): task broken into stages, multiple tasks spread across stages → different task in each stage at once

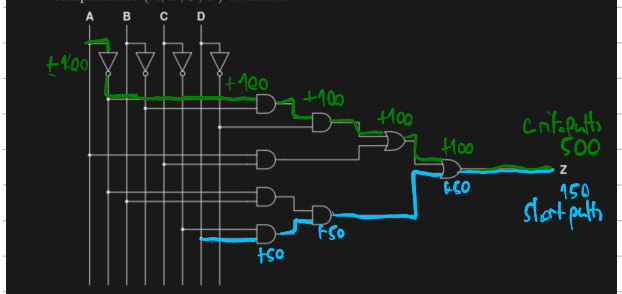
→ assume a latency L for a nonparallel task with a throughput of 1/L. With N copies of HW, it becomes N/L, which is the ideal case for temporal par. too, but in practice, you get $1/L_1$ ← longest latency for a step

→ Exam practicality: Prof. Mathu hasn't made a timing task yet, but they do come up in older exams

Nice trivial question! (FS15), we're looking at e)

→ in general, I would focus on the basics, less on skew/synchronizers

(d) (4 points) Draw a gate-level schematic that realizes the function Z using **only** 2-input AND, OR gates. Assume that you have all the inputs (A, B, C, D) and their complements $(\overline{A}, \overline{B}, \overline{C}, \overline{D})$ available.



(e) (2 points) Assume that all the gates (AND, OR, NOT) in the previous diagram have a propagation delay of 100 ps and a contamination delay of 50 ps. What is the delay of the longest (*critical*) path and the *shortest* path of this circuit?

next section: Verilog!

→ this may be asked: Verilog is not a programming language, it is a HDL (hardware description language) to allow for synthesis of digital circuits, but it often features similar syntax

→ a module is a HW block that needs defined inputs/outputs

→ Behavioral Verilog: using logical operators to describe what a module does

→ Structural Verilog: gate-level description at the bottom level, at a higher level it is about building modules using instances of modules

→ you start with name and listing I/O

→ the assign LHS = RHS statement is used for CL outside always statements

→ ~ → NOT, & = AND, | → OR, ~& → NAND, ~| → NOR, ^ → XOR, ~^ → XNOR

→ Verilog signals (individual bits of variables) can be set to 0, 1, Z (floating), X (unknown)

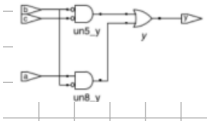
```
module sillyfunction (input a, b, c,
                      output y);

  assign y = ~a & ~b & ~c |
             a & ~b & ~c |
             a & ~b &  c;

endmodule
```

Verilog Operator	Name	Functional Group
[ ]	bit-select or part-select	
( )	parenthesis	
!	logical negation	logical
~	negation	bit-wise
&	reduction AND	reduction
|	reduction OR	reduction
~&	reduction NAND	reduction
~|	reduction NOR	reduction
^	reduction XOR	reduction
~^ or ^~	reduction XNOR	reduction
+	unary (sign) plus	arithmetic
-	unary (sign) minus	arithmetic
{ }	concatenation	concatenation
{{ }}	replication	replication
*	multiply	arithmetic
/	divide	arithmetic
%	modulus	arithmetic
+	binary plus	arithmetic
-	binary minus	arithmetic
<<	shift left	shift
>>	shift right	shift
>	greater than	relational
>=	greater than or equal to	relational
<	less than	relational
<=	less than or equal to	relational
==	case equality	equality
!=	case inequality	equality
&	bit-wise AND	bit-wise
^	bit-wise XOR	bit-wise
|	bit-wise OR	bit-wise
&&	logical AND	logical
||	logical OR	logical
?:	conditional	conditional

→ Simulation: checking modules for logical correctness

→ Synthesis: HDL code is transformed to a netlist describing hardware produced

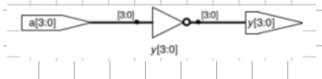
→ code may be simplified during synthesis, there is a schematic output like so:



→ a top module is the main piece of hardware, can consist of CL blocks, registers, FSMs

→ CL: bitwise operators act on single-bit signals or multi-bit busses

```
module inv (input  [3:0] a,
            output [3:0] y);

  assign y = ~a;
endmodule
```



a[3:0] ← 4-bit bus, given in little-endian order (a[3] is MSB, a[0] LSB)

a[0:3] ← big-endian, also permitted

→ just be consistent

```
module gates(input  [3:0] a, b,
             output [3:0] y1, y2,
                          y3, y4, y5);

  /* Five different two-input logic
     gates acting on 4 bit busses */
  assign y1 = a & b;    // AND
  assign y2 = a | b;    // OR
  assign y3 = a ^ b;    // XOR
  assign y4 = ~(a & b); // NAND
  assign y5 = ~(a | b); // NOR
endmodule
```

→ you know the code, the assign statements we saw are continuous, end with ;

→ when inputs on RHS change, LHS recomputed

→ Verilog comments are exactly like Java

→ white space is irrelevant, module names can't begin with numbers, vars. are case-sensitive

reduction operators → using the operators we just introduced in a unary way

```
module and8(input [7:0] a,
            output      y);

  assign y = &a;

  // &a is much easier to write than
  // assign y = a[7] & a[6] & a[5] & a[4] &
  //            a[3] & a[2] & a[1] & a[0];
endmodule
```

conditional assignment: you know this as the ternary operator →

```
module mux4(input  [3:0] d0, d1, d2, d3,
            input  [1:0] s,
            output [3:0] y);

  assign y = s[1] ? (s[0] ? d3 : d2)
                  : (s[0] ? d1 : d0);
endmodule
```

(i did this for Lab 5)

```
module fulladder(input  a, b, cin,
                 output s, cout);

  wire p, g;

  assign p = a ^ b;
  assign g = a & b;

  assign s = p ^ cin;
  assign cout = g  |  (p & cin);
endmodule
```

boolean equations for this full adder: $P = A \text{ XOR } B$, $G = A \cdot B$, $S = P \text{ XOR } C_{in}$, $C_{out} = G + P \cdot C_{in}$

→ the continuous assign statements are concurrent → for CL, order doesn't matter

→ wires are declared to hold internal vars. like P and G, they only need to be declared for multi-bit busses but are good practice

Numbers: (the book won't tell you this: variables can have the data types integer and real → Example: real seven = 7.00;)

Generally: number format: N'b value, bases: 'b → binary, 'o → octal, 'd → decimal, 'h → hex (N: size in bits, b: base)

→ size can be omitted, then a number is assumed to have as many bits as its expression

Table 4.5 Verilog AND gate truth table with z and x ← you can see x as "should not happen"

&	A: 0	A: 1	A: z	A: x
B: 0	0	0	0	0
B: 1	0	1	x	x
B: z	0	x	x	x
B: x	0	x	x	x

```
module tristate(input  [3:0] a,
                input        en,
                output [3:0] y);

  assign y = en ? a : 4'bz;
endmodule
```

you can push z, x →

bit Swizzling: `assign y = {c[2:1], {3{d[0]}}, c[0], 3'b101};`

→ { } concatenates busses and bits (and constants) → example: {3{d[0]}} copies d[0] thrice

Delays: ignored in synthesis, used in simulation and testing → ensure timing is correct

→ of course, electrical factors in gate delays cannot be considered

```
`timescale 1ns/1ps

module example(input  a, b, c,
               output y);
  wire ab, bb, cb, n1, n2, n3;

  assign #1 {ab, bb, cb} = ~{a, b, c};
  assign #2 n1 = ab & bb & cb;
  assign #2 n2 = a & bb & cb;
  assign #2 n3 = a & bb & c;
  assign #4 y = n1 | n2 | n3;
endmodule
```

→ timescale is necessary in a file i believe, format: `timescale unit/precision

→ here: every unit is 1ns, sim. has 1ps precision

→ # indicates number of units of delay → here: and → 2ns delay, or → 4ns delay

→ can be placed in assign, non-blocking, blocking assignments

Structural modeling:

→ the book actually employs bad practice for instantiation:

↔ good practice: submodule name (.input1(i1), .output1(o1));
example → mux2 lowmux(.d0(d0), .d1(d1), .s(s[0]), .y(low));

```
module mux2(input  [3:0] d0, d1,
            input        s,
            output [3:0] y);
  tristate t0(d0, s, y);
  tristate t1(d1, s, y);
endmodule
```

you can operate in instantiations

```
module mux4(input  [3:0] d0, d1, d2, d3,
            input  [1:0] s,
            output [3:0] y);

  wire [3:0] low, high;

  mux2 lowmux(d0, d1, s[0], low);
  mux2 highmux(d2, d3, s[0], high);
  mux2 finalmux(low, high, s[1], y);
endmodule
```

↳ 3 instances of mux2

→ accessing parts of busses: use array notation following your endian format → d0[7:4] (example)

```
module mux2_8(input  [7:0] d0, d1,
              input        s,
              output [7:0] y);

  mux2 lsbmux(d0[3:0], d1[3:0], s, y[3:0]);
  mux2 msbmux(d0[7:4], d1[7:4], s, y[7:4]);
endmodule
```

Sequential Blocks:

```
module flop (input            clk,
             input      [3:0] d,
             output reg [3:0] q);

  always @ (posedge clk)
    q <= d;
endmodule
```

→ register logic is performed using always statements (always @ (sensitivity list);)

→ signals keep their values unless an event in the sensitivity list occurs

→ usually it is a clock, as in this case → posedge: rising edge, negedge: falling edge

→ we don't use assign statements here, those are for CL blocks outside of always statements

→ instead, we use the nonblocking assignment (<=), which is used for seq. logic and runs concurrently (no order) unlike =

→ all signals on the LHS of always statements must be declared a register → example: output reg [3:0] q

example: resettable register, allowing for both sync. and async. reset (2 modules)

```
module flopr (input            clk,
              input            reset,
              input      [3:0] d,
              output reg [3:0] q);

  // asynchronous reset
  always @ (posedge clk, posedge reset)
    if (reset) q <= 4'b0;
    else       q <= d;
endmodule

module flopr (input            clk,
              input            reset,
              input      [3:0] d,
              output reg [3:0] q);

  // synchronous reset
  always @ (posedge clk)
    if (reset) q <= 4'b0;
    else       q <= d;
endmodule
```

→ multiple elements in an always sens. list are separated by a comma or "or"

→ you will see, we have if statements, for loops, while loops ...

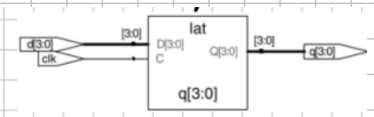
synchronizer:

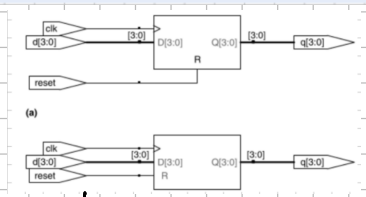
```
module sync (input      clk,
             input      d,
             output reg q);

  reg n1;

  always @ (posedge clk)
    begin
      n1 <= d;
      q <= n1;
    end
endmodule
```

→ begin and end are necessary in any statement or block that makes more than one assignment, good practice otherwise

Latch:

```
module latch (input            clk,
              input      [3:0] d,
              output reg [3:0] q);

  always @ (clk, d)
    if (clk) q <= d;
endmodule
```





→ always @ (clk, d) recomputes statements inside whenever clk or d changes

→ always @ (*) recomputes the statements inside whenever any signal on RHS of = or <= changes

→ blocking assignment (=) used for CL in always blocks, they run sequentially in-order

→ always blocks mostly for sequential blocks, but can behaviorally describe CL if the sens. list responds to all input changes and the body gives outputs in all cases

→ the book calls always blocks "always/process statements"

→ case and if can only be used in always or initial blocks (initial does not synthesize and is used for testbenches)

Verilog

```
module inv (input      [3:0] a,
            output reg [3:0] y);

  always @ (*)
    y = ~a;
endmodule
```

```
module sevenseg (input      [3:0] data,
                 output reg [6:0] segments);
  always @ (*)
    case (data)
      //                abc_defg
      0: segments = 7'b111_1110;
      1: segments = 7'b011_0000;
      2: segments = 7'b110_1101;
      3: segments = 7'b111_1001;
      4: segments = 7'b011_0011;
      5: segments = 7'b101_1011;
      6: segments = 7'b101_1111;
      7: segments = 7'b111_0000;
      8: segments = 7'b111_1111;
      9: segments = 7'b111_1011;
      default: segments = 7'b000_0000;
    endcase
endmodule
```

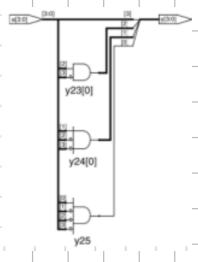
→ case checks value of an input (here data) in base 10

→ if-else statements can be made like in regular programming, just follow the begin/end rules

→ casez includes don't cares from ??'s as ?

→ rule in always blocks: don't assign to the same signal more than once

```
module priority_casez(input      [3:0] a,
                      output reg [3:0] y);

  always @ (*)
    casez (a)
      4'b1???: y = 4'b1000;
      4'b01??: y = 4'b0100;
      4'b001?: y = 4'b0010;
      4'b0001: y = 4'b0001;
      default: y = 4'b0000;
    endcase
endmodule
```



sign extend:

FSM:

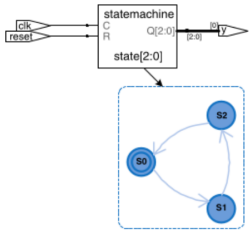
Verilog (divides clk by 3)

```
module divideby3FSM(input  clk,
                    input  reset,
                    output y);

  reg [1:0] state, nextstate;

  parameter S0 = 2'b00;
  parameter S1 = 2'b01;
  parameter S2 = 2'b10;

  // state register
  always @ (posedge clk, posedge reset)
    if (reset) state <= S0;
    else       state <= nextstate;

  // next state logic
  always @ (*)
    case (state)
      S0: nextstate = S1;
      S1: nextstate = S2;
      S2: nextstate = S0;
      default: nextstate = S0;
    endcase

  // output logic
  assign y = (state == S0);
endmodule
```

→ parameter statements are used to define constants and are standard for any FSM state

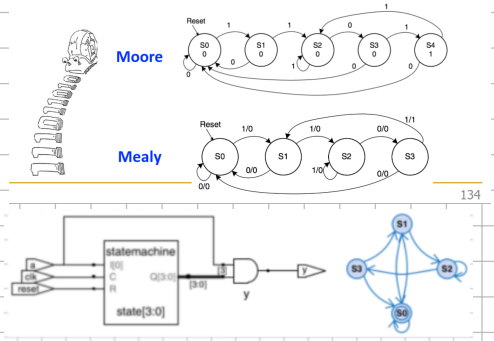
→ default in case needed to fulfill our def. of combinational for next state logic

→ notice the equality comparison == or inequality != with strict versions === and !==



FSM Example 2: Recall the Smiling Snail

- Alyssa P. Hacker has a snail that crawls down a paper tape with 1's and 0's on it
- The snail smiles whenever the last four digits it has crawled over are 1101
- Design Moore and Mealy FSMs of the snail's brain



Verilog (Moore-style)

```
module patternMoore(input  clk,
                    input  reset,
                    input  a,
                    output y);

  reg [2:0] state, nextstate;

  parameter S0 = 3'b000;
  parameter S1 = 3'b001;
  parameter S2 = 3'b010;
  parameter S3 = 3'b011;
  parameter S4 = 3'b100;

  // state register
  always @ (posedge clk, posedge reset)
    if (reset) state <= S0;
    else       state <= nextstate;

  // next state logic
  always @ (*)
    case (state)
      S0: if (a) nextstate = S1;
          else   nextstate = S0;
      S1: if (a) nextstate = S2;
          else   nextstate = S0;
      S2: if (a) nextstate = S2;
          else   nextstate = S3;
      S3: if (a) nextstate = S4;
          else   nextstate = S0;
      S4: if (a) nextstate = S2;
          else   nextstate = S0;
      default:   nextstate = S0;
    endcase

  // output logic
  assign y = (state == S4);
endmodule
```

Verilog (Mealy style)

```
module patternMealy(input  clk,
                    input  reset,
                    input  a,
                    output y);

  reg [1:0] state, nextstate;

  parameter S0 = 2'b00;
  parameter S1 = 2'b01;
  parameter S2 = 2'b10;
  parameter S3 = 2'b11;

  // state register
  always @ (posedge clk, posedge reset)
    if (reset) state <= S0;
    else       state <= nextstate;

  // next state logic
  always @ (*)
    case (state)
      S0: if (a) nextstate = S1;
          else   nextstate = S0;
      S1: if (a) nextstate = S2;
          else   nextstate = S0;
      S2: if (a) nextstate = S2;
          else   nextstate = S3;
      S3: if (a) nextstate = S1;
          else   nextstate = S0;
      default:   nextstate = S0;
    endcase

  // output logic
  assign y = (a & state == S3);
endmodule
```

Parametrizing:

Verilog

```
module mux2
  #(parameter width = 8)
   (input  [width-1:0] d0, d1,
    input              s,
    output [width-1:0] y);
  assign y = s ? d1 : d0;
endmodule
```

default value

↑ depends on width

we use passing a variable to decide module's width

```
module mux4_8(input  [7:0] d0, d1, d2, d3,
              input  [1:0] s,
              output [7:0] y);

  wire [7:0] low, hi;

  mux2 lowmux(d0, d1, s[0], low);
  mux2 himux(d2, d3, s[1], hi);
  mux2 outmux(low, hi, s[1], y);
endmodule
```

→ you can instantiate like normal - 8 is default

```
module mux4_12(input  [11:0] d0, d1, d2, d3,
               input  [1:0]  s,
               output [11:0] y);

  wire [11:0] low, hi;

  mux2 #(12) lowmux(d0, d1, s[0], low);
  mux2 #(12) himux(d2, d3, s[1], hi);
  mux2 #(12) outmux(low, hi, s[1], y);
endmodule
```

→ need to instantiate with parameter 12

Testbenches: a HDL module used to test another module, called Device Under Test (DUT)

→ inputs and expected outputs → test vectors

Simple testbenches: instantiate DUT, apply inputs in initial block, use blocking assignments and delays, then the user manually verifies correct outputs → simulated like any module, not synthesizable

→ example: sillyfunction is a simple 3-input logic function → 8 test vectors

```
module testbench1();
  reg  a, b, c;
  wire y;

  // instantiate device under test
  sillyfunction dut(a, b, c, y);

  // apply inputs one at a time
  initial begin
    a = 0; b = 0; c = 0; #10;
    c = 1;               #10;
    b = 1; c = 0;        #10;
    c = 1;               #10;
    a = 1; b = 0; c = 0; #10;
    c = 1;               #10;
    b = 1; c = 0;        #10;
    c = 1;               #10;
  end
endmodule
```